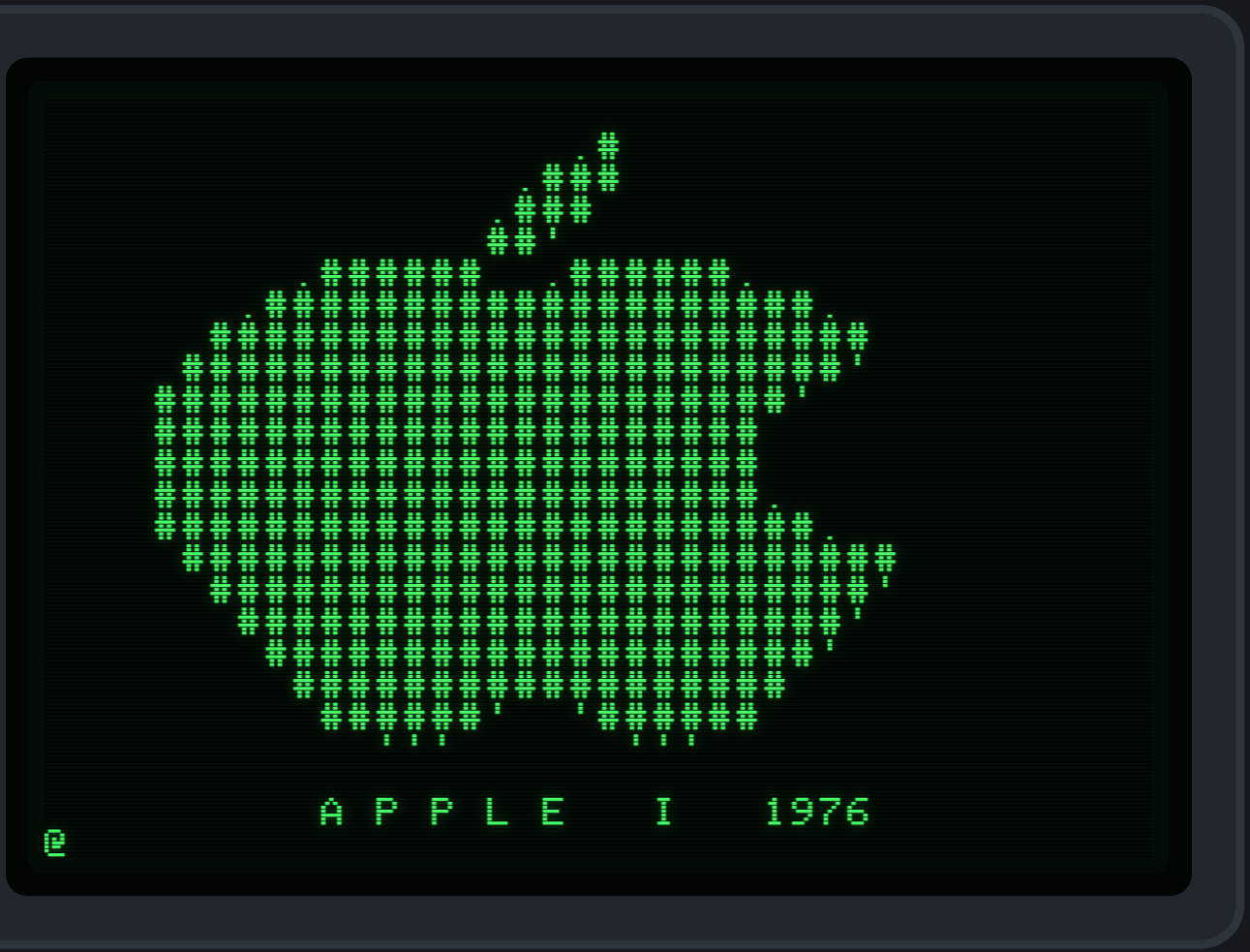
RETRO DEV KIT APPLE 1 — THE ORIGINAL SIN

Quick Start Guide

Install the emulator, load a tape, write a first BASIC program and send it to a real Apple-1



VERSION 1.0 NOVEMBER 2026 ENGLISH EDITION

Contents

1 Welcome 3
- 1.1 What you need 3
- 1.2 How to read this guide 3

2 Installing on Linux 5
- 2.1 Unpack and run 5
- 2.2 If the window does not open 6
- 2.3 Where your files are kept 6

3 First launch 7
- 3.1 The Woz Monitor 8
- 3.2 Your first machine-language program 9

4 Loading and running a tape 10
- 4.1 Preparing the library 10
- 4.2 The quickest way: the ▶ button 10
- 4.3 As in 1976: the virtual tape 11

5 Your first BASIC program 13
- 5.1 Writing the program 13
- 5.2 Running it 13
- 5.3 When there is an error 14

6 On a real Apple-1: exporting to WAV 16
- 6.1 Making the tape 16
- 6.2 Loading on the real machine 16
- 6.3 Why AIFF at 48 kHz 17

7 What next? 18
- 7.1 Essential shortcuts 18
- 7.2 Commands to remember 19

CHAPTER 1

Welcome

Retro Dev Kit Apple 1 — *"The Original Sin" — brings Steve Wozniak's Apple-1 (1976) back to life on a Linux PC: the 6502 processor, the 40 × 24 character display, the Woz Monitor, the ACI cassette interface and a built-in Integer-BASIC-compatible BASIC (RDK G2 Basic). Above all, it makes audio tapes that you can play into a real Apple-1.*

In about half an hour, this guide takes you from the downloaded archive to a BASIC program of your own, loaded into a real machine, with a screenshot at every step. The **Complete Manual** covers every panel in detail, with full guides to Integer BASIC and the 6502 assembler.

1.1 What you need

A PC running Linux

A recent Linux distribution, 64-bit (x86_64) or 32-bit (i686). Nothing to install: the emulator uses the graphics libraries already there (X11 or Wayland, OpenGL/Mesa).

The `apple1-emu.zip` archive

Both programs (window and terminal), examples, this guide and the Complete Manual in PDF.

BASIC is included

RDK G2 Basic, a free BASIC compatible with Integer BASIC, is built in. Apple's 1976 original is not included (still under copyright): if you want it, use your own copy, such as the `BASIC.wav` tape of the archive.org "Apple1Cassettes" collection.

Optional: a real Apple-1

With an ACI cassette card: original, replica or kit. A smartphone or a PC headphone output is the tape player.

1.2 How to read this guide

Commands to type on the Apple-1 are written `LIKE THIS`. Sessions appear on a black background, like the screen of the time; what you type is white, the machine's answer green:

WOZ MONITOR

```
300R
0300: A9A
```

Three kinds of boxes appear along the way:

> ✦ **TIP**
>
> A shortcut, or a quicker way to get the same result.

> ! **CAUTION**
>
> A common trap, or an action that can lose your work.

> ▣ **ON THE REAL APPLE-1**
>
> What changes when you move from the emulator to the real machine.

CHAPTER 2

Installing on Linux

2.1 Unpack and run

1. **Unpack the archive.** Right-click `apple1-emu.zip`, then "Extract Here". An `apple1-emu` folder appears.
2. **Double-click `run.sh`.** If the file manager asks what to do with the file (for example on Linux Mint), choose **"Run"** (not "Display"). The Apple-1 window opens, without a terminal.
3. **Optional: add the emulator to the menu.** Open a terminal in the folder (right-click → "Open in Terminal") and type the command below. A **Retro Dev Kit Apple 1** entry appears in the *Games* section of the menu, together with the commands `apple1-emu-gui` (window) and `apple1-emu` (terminal).

TERMINAL

```
./install.sh             # applications menu + commands in ~/.local/bin
./install.sh --bureau    # the same, plus a desktop icon
./install.sh --desinstaller
```

Uninstalling (`--desinstaller`) removes the programs and the menu entry but keeps your programs (`~/apple1/programmes`) and your settings (`~/.config/apple1-emu`). No command needs `sudo`: everything is installed in your home folder. The first launch of a new version offers to start again from the original settings: the library is kept. The **Reset preferences** button does the same. An empty file named `portable` next to the program (or `APPLE1_PORTABLE=1`) keeps preferences, the default library and the cache inside the kit folder.

✦ UPDATING FROM AN OLDER VERSION

The `lancer.sh` and `installer.sh` scripts of earlier versions are now called `run.sh` and `install.sh`. Run `./install.sh` again: it replaces the old installation, and the menu entry and desktop icon that pointed to `lancer.sh`.

✦ THE TERMINAL VERSION

`./run.sh --terminal` opens the emulator in a terminal (at least 42 columns × 28 lines), without a graphical interface. Handy over an SSH connection. The Complete Manual describes its keys and commands.

2.2 If the window does not open

`run.sh` detects an immediate failure (graphics driver, virtual machine without acceleration) and retries on its own with Mesa software rendering (`LIBGL_ALWAYS_SOFTWARE=1`). If that fails too, a message is shown and the log is in `~/.cache/apple1-emu/gui.log` .

Symptom	What to do
The file manager opens `run.sh` in the text editor	Right-click → Properties → Permissions: tick “Allow executing file as program”, then double-click again and choose “Run”.
“Permission denied” in the terminal	`chmod +x run.sh install.sh bin/*/apple1-*`
Black or very slow window	Normal in a virtual machine: software rendering stays smooth for the Apple-1 screen. Turn off the “Vintage screen” if needed.
No tape sound	Sound goes through ALSA; without a sound card it is simply muted. Loading still works.

2.3 Where your files are kept

Location	Contents
`~/apple1/programmes`	The **library**: your tapes, programs and examples (created on first launch).
`~/.config/apple1-emu/gui.json`	Settings: zoom, colors, panel layout, editor tabs.
`~/.cache/apple1-emu/`	Converted tapes, downloaded GitHub projects, the `gui.log` log.

CHAPTER 3

First launch

On first launch the window opens maximized, and a **Your workspace** window offers five layouts: **Default**, **Game**, **BASIC**, **Assembly** and **Debugging**. Each numbered thumbnail shows the layout, says what it is for and gives the minimum screen size it needs: click the one you want. Untick **Show this window at start-up** if you prefer not to see it again; **⊞ Layout ▾ → Choose a workspace...** opens it again. You can change the layout at any time with the **⊞ Layout ▾** menu. The default layout makes every tool visible at a glance. Each area is a dockable panel that you can move, group into tabs or detach into its own window. The colour screen (GEN2 / P-LAB) is rectangle number 7, in the thumbnails that show it and in the legend; numbers 1 to 6 do not change. This window does not stay above the others. If the window is narrow, the toolbar scrolls: the **»** button brings the Cassette group.

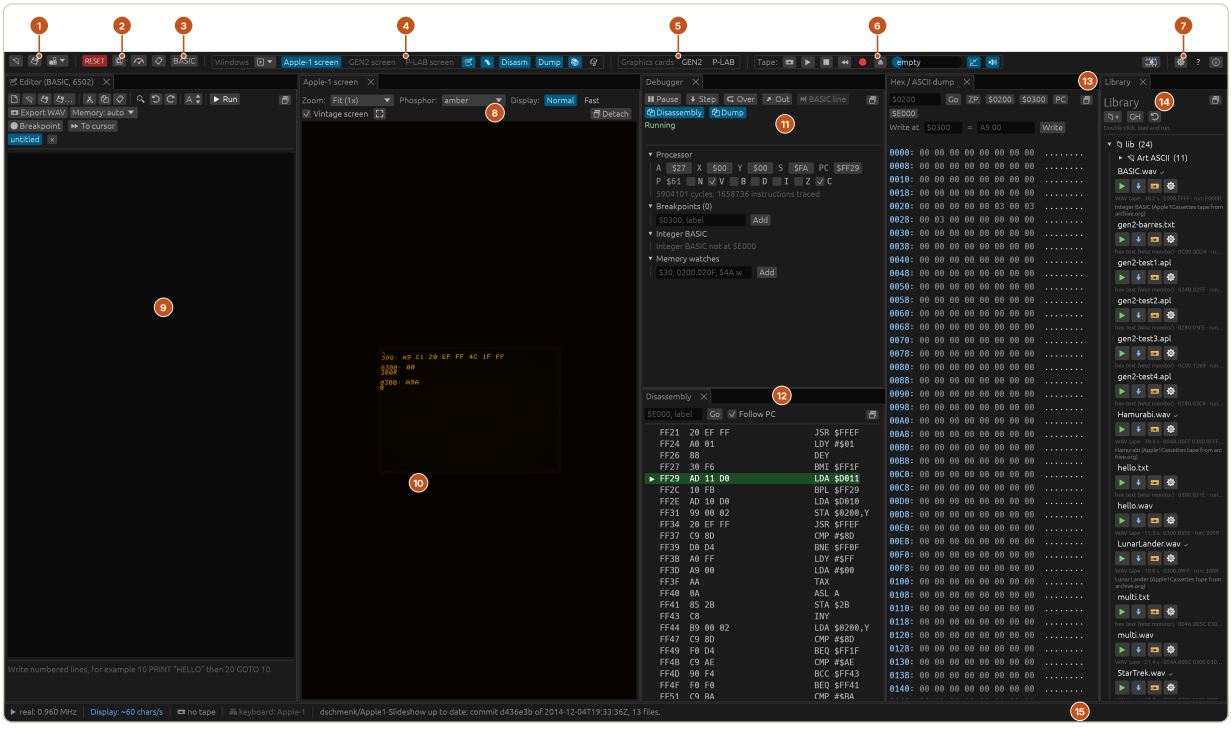


Figure 1 — The default layout. The screen already shows the small program from the next section.

1. **Load, Save, State**: `.txt` hex dumps, `.bin` and `.prg` binaries; Save also writes `.wav` / `.aiff` tapes; State saves and restores the machine.
2. **RESET, Clear screen, Speed (gauge), Paste**: the machine's buttons (Ctrl+R, Ctrl+L, F3, Ctrl+V).
3. **BASIC**: loads the built-in RDK G2 Basic at `$E000`.
4. **Windows**: the **⊞ Layout ▾** menu, then one button per panel (screens, Editor, Debugger, Disassembly, Dump, Library, ASCII).
5. **Graphics cards**: **GEN2** and **P-LAB** plug the cards in or out.
6. **Tape**: **Insert** a tape, then play, stop, rewind, record, eject; oscilloscope and sound.

7. **EPROM**, ⚙ **Preferences**, **?** (detailed tooltips, also F1) and **i** (about, shortcuts).
8. **Display row of the screen**: zoom (1x to 8x or Fit), phosphor, display speed **Normal** or **Fast**, vintage screen, full screen, **Detach**.
9. **Editor**: BASIC programs and 6502 sources in tabs.
10. **Apple-1 screen**: 40 × 24 characters; click it to type, right-click for Copy, Paste, Clear screen and Reset.
11. **Debugger**: pause, single step, registers, breakpoints, variables.
12. **Disassembly**: the code around the program counter (PC).
13. **Dump**: memory in hex and ASCII, editable.
14. **Library**: your programs, each with its small icons ▶ ⬇ 📼 ⚙.
15. **Status bar**: speed, display rate, tape, keyboard target; below it, the last message.

> ✦ **MAKING ROOM**
>
> To just play, close the editor and debugger with their buttons in the Windows group (4): the screen grows. In the zoom list (8), **Fit** picks the largest whole zoom that fits, **F11** goes full screen, ⊞ **Layout** ▾ → **Default** (4) restores everything; the same menu also offers a **Game** layout (Apple-1 screen and library). If the buttons are too small or too large for your screen, change ⚙ **Preferences** → **Interface** → **Icon and text size**.

> **NETWORK ACCESS AT FIRST LAUNCH**
>
> The first time it starts, the emulator contacts GitHub (`api.github.com`, `raw.githubusercontent.com`) to fetch the demo project (`dschmenk/Apple1-Slideshow`, listed under "GitHub projects" in ⚙ **Preferences**), and shows its progress in the status bar. Nothing else is sent. Offline, it simply skips it; remove the project from that list to stop the download for good.

3.1 The Woz Monitor

The backslash `\` is the prompt of the **Woz Monitor**, the 256-byte program that starts the Apple-1. Click the screen and type: lower case becomes upper case, **Enter** is RETURN, **Backspace** gives the `_` that rubs out the last character, as in 1976.

The monitor understands four kinds of commands:

You type	Effect
`300`	shows the byte at address `$0300`
`300.30F`	shows the bytes from `$0300` to `$030F`
`300: A9 C1`	writes `A9` at `$0300`, then `C1` at `$0301`
`300R`	runs the program that starts at `$0300` (*Run*)

3.2 Your first machine-language program

Type these two lines, each followed by Enter:

WOZ MONITOR

```
300: A9 C1 20 EF FF 4C 1F FF

0300: 00
300R

0300: A9A
```

The final `A` was printed by your program. The eight bytes are three 6502 instructions:

Bytes	Instruction	Purpose
`A9 C1`	`LDA #$C1`	loads the code of the letter “A” (with bit 7 set, as the Apple-1 expects)
`20 EF FF`	`JSR $FFEF`	calls ECHO, the monitor routine that prints one character
`4C 1F FF`	`JMP $FF1F`	returns to the monitor, which waits for the next command

The monitor first shows the byte at the start address (`0300: A9`), then your program prints the `A` . The built-in assembler (Complete Manual) saves you from working out these bytes by hand.

> ▣ **ON THE REAL APPLE-1**
>
> The commands are identical: the emulator runs the real Woz Monitor ROM. The real machine displays about 60 characters per second. The emulator keeps that pace when the display row of the screen is set to **Normal**; **Fast** shows everything at once. The setting is shared by the Apple-1 screen and the GEN2 screen.

CHAPTER 4

Loading and running a tape

On the Apple-1, everything goes through the cassette: Integer BASIC itself was loaded from an audio tape. The emulator reads `.wav` and `.aiff` files the way the original ACI card does, byte by byte, and recognizes the 24 tapes of the "Apple1Cassettes" collection on archive.org.

4.1 Preparing the library

Copy your tapes into the library folder, `~/apple1/programmes` : for instance `Hamurabi.wav`. The list on the right updates by itself. The five folders BASIC, 6502, GEN2 COLOR, WAV and Art ASCII are listed there directly, with no root folder above them; files left loose in the library folder are not shown. Each recognized tape is marked ↻ **recognised**, with its address ranges and run command already set.

> **YOU DO NOT NEED BASIC.WAV**
>
> The emulator has its own BASIC, RDK G2 Basic, so BASIC programs run without any tape. Apple's 1976 Integer BASIC is still under copyright and is not included: if you prefer the original, use your own copy (such as `BASIC.wav`) and choose it in ⚙ **Preferences** → **Advanced** → **BASIC**.

4.2 The quickest way: the ▶ button

Each program in the list has four small icons on its right: ▶ loads and runs, ⬇ loads into memory without running, 📼 loads through the tape, ⚙ opens its settings. A double-click does the same as ▶.

Click ▶ next to `Hamurabi.wav`. The emulator loads BASIC if it is not already in memory, then the program, and types `E2B3R` (back to BASIC without erasing the program) and `RUN` for you. Loading is instant.

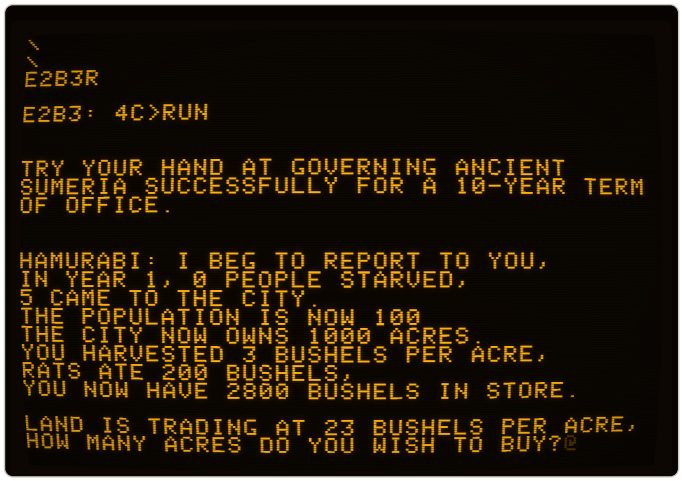


Figure 2 — Hamurabi started with the ▶ button of the library.

For a machine-language program, ▶ does a RESET and then types its run command, for example `300R`. For a BASIC listing or a 6502 source, ▶ also opens it in a tab of the editor, if the editor is open.

4.3 As in 1976: the virtual tape

Loading is instant unless you ask for the tape. Click the 📼 icon of a program to reproduce a real load: the emulator types `C100R` to start the ACI program, then the address range, and the tape plays at real speed. A progress bar appears under the program's name, and the **Tape:** strip of the toolbar shows the tape and its position.

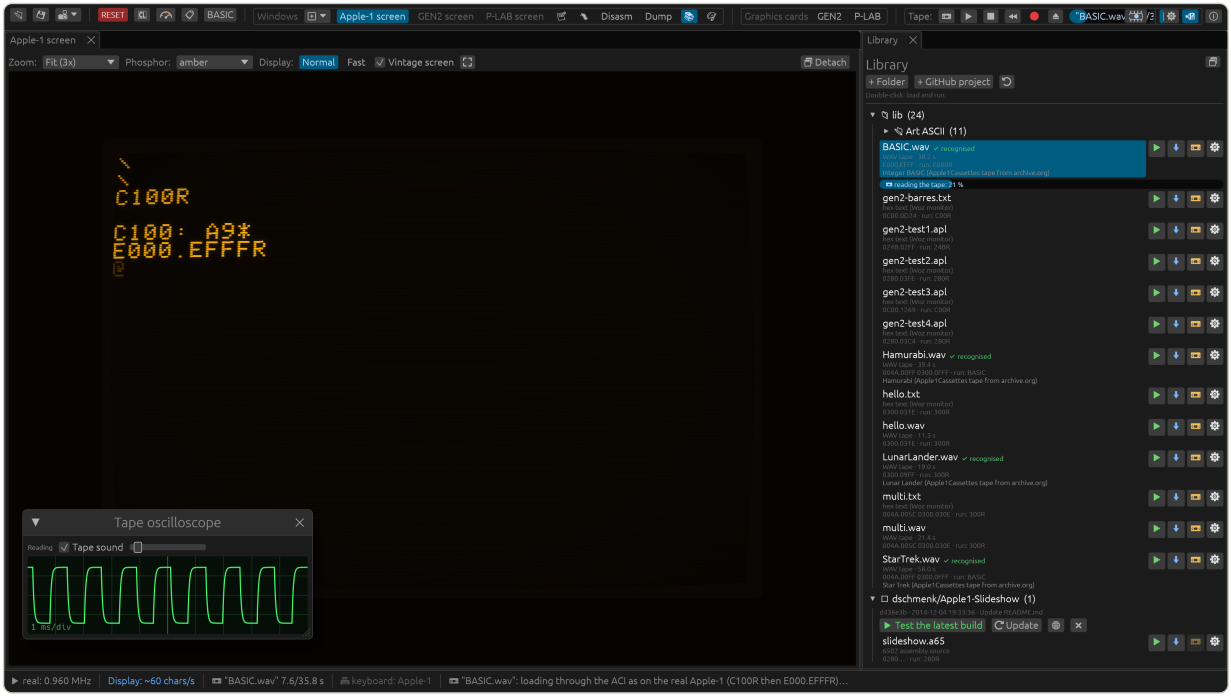


Figure 3 — Loading BASIC.wav through the virtual ACI, with the tape oscilloscope open.

The 📈 and 🔊 buttons at the end of the **Tape:** strip open the **oscilloscope** and let you **hear** the signal, like a tape recorder's speaker. Loading BASIC takes about 36 seconds at real speed.

When the tape stops, the monitor shows `\` again. Type `E000R` to start BASIC. Its prompt is a `>` :

WOZ MONITOR

```
E000R

E000: 4C
>PRINT 6*7
42
```

✦ LOADING BY HAND

📼 **Insert**, in the **Tape:** strip of the toolbar, puts any `.wav` or `.aiff` file in the virtual deck. You then type `C100R` yourself, then the range, for example `E000.EFFFR` for BASIC: the tape starts at the first read, as on the real machine. The **Load** button is for `.txt` , `.bin` and `.hex` files, loaded instantly.

Program	After loading
Integer BASIC	`E000R` (cold start, erases the program in memory)
BASIC program	`E2B3R` (warm start, keeps the program), then `RUN`
Machine language	its address followed by `R` , often `300R`

CHAPTER 5

Your first BASIC program

Integer BASIC, Steve Wozniak's BASIC, fits in 4 KB, and the emulator's own **RDK G2 Basic**, which runs the same programs, is built in. It only knows integers (−32767 to 32767), but it is fast and pleasant. The built-in editor spares you typing on the Apple-1 screen, where a typo means retyping the whole line.

5.1 Writing the program

Click 📝 **Editor** in the toolbar, then the **New** icon (the first one) of the editor and choose "BASIC program", and type this program (also in `docs/exemples/en/basic/premier.bas`).

INTEGER BASIC

premier.bas

```
10 REM MY FIRST PROGRAM
20 PRINT "HELLO, APPLE-1!"
30 PRINT
40 FOR I=1 TO 8
50 PRINT I;" SQUARED = ";I*I
60 NEXT I
70 PRINT
80 DIM N$(20)
90 INPUT "YOUR FIRST NAME",N$
100 PRINT "NICE TO MEET YOU, ";N$;"!"
110 END
```

Keywords, numbers, strings and comments each get their own color (the colors can be changed in ⚙ **Preferences** → **Editor**); below the text, the editor shows the program size and the memory left for variables.

✦ RIGHT-CLICK IN THE TEXT

The editor's menu holds cut, copy and paste, **Find** (Ctrl+F) and **Replace** (Ctrl+H), go to a line (Ctrl+G), REM / un-REM a block, **Renumber…** (`GOTO`, `GOSUB` and `THEN` follow), a breakpoint on the line (F9) and **Send and run** (F5).

5.2 Running it

Click ▶ **Run**. The program goes straight into memory (after BASIC, if needed), then the editor types `E2B3R` and `RUN` for you:

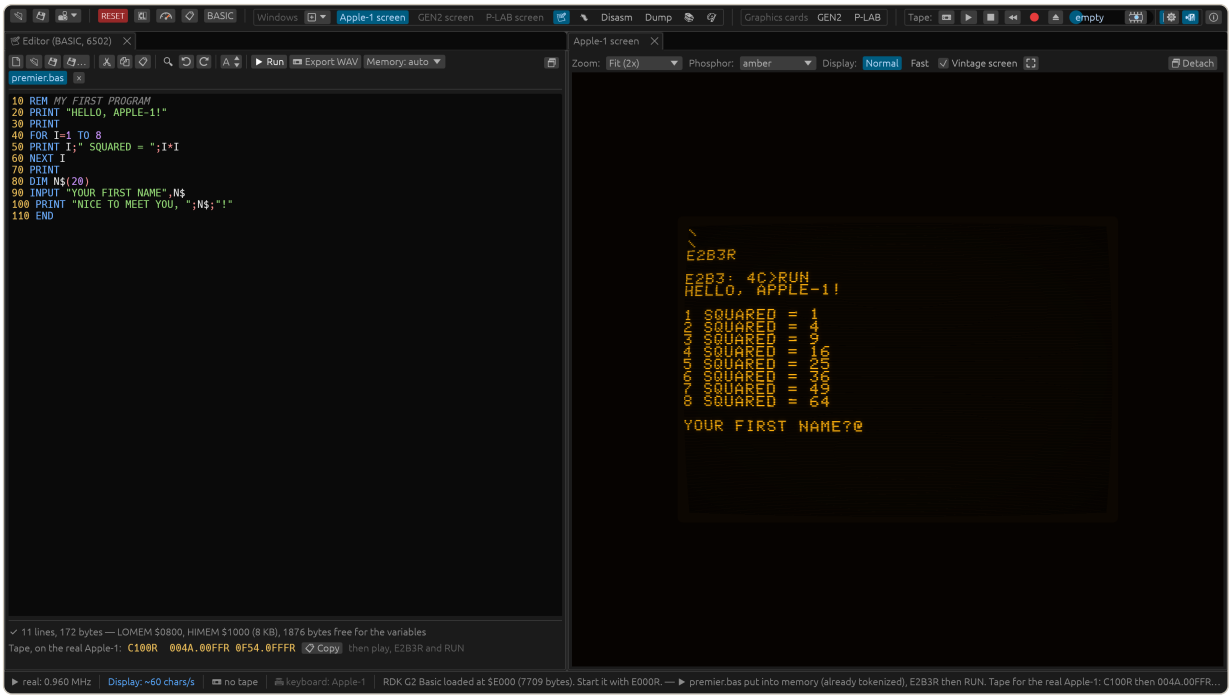


Figure 4 — The editor on the left, the running program on the right.

The program asks for your first name. Click the Apple-1 screen, type it, then Enter:

```
YOUR FIRST NAME?WOZ
NICE TO MEET YOU, WOZ!
```
WOZ MONITOR

! THREE QUIRKS OF INTEGER BASIC

- **Strings must be dimensioned**: without `DIM N$(20)`, the `INPUT` of a string stops with `*** STR OVFL ERR`.
- **Variable names** are one letter, optionally followed by a digit: `A`, `B2`, `N$`. `TOTAL` is a syntax error.
- **The final `END` is mandatory**: without it, the program ends with `*** END ERR`.

5.3 When there is an error

The editor checks each line as you type, with the rules of the real BASIC. A faulty line turns red and the message explains the problem, with the line number and column:

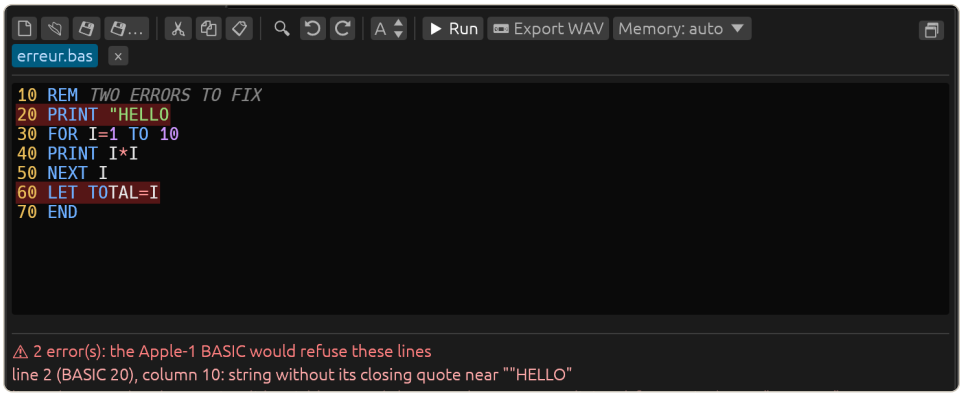


Figure 5 — Two errors reported: an unterminated string and a variable name that is too long.

A program with errors does not run: fix the red lines first. Run-time errors appear on the Apple-1 screen: `*** BAD BRANCH ERR` for a `GOTO` to a missing line, `*** >32767 ERR` for an overflow or a division by zero. The **Debugger** (Complete Manual) then steps line by line while watching the variables.

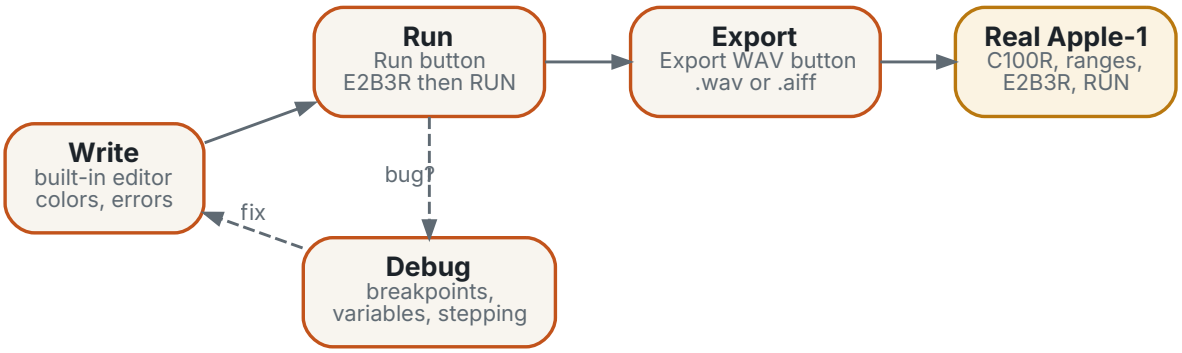


Figure 6 — The work cycle: write, run, fix, then export to the real machine.

> ✦ **STOPPING A PROGRAM**
>
> During a `RUN`, a key press stops the program with `STOPPED AT 50` (the current line). If it is stuck, **Reset** (Ctrl+R) returns to the monitor and `E2B3R` re-enters BASIC **without erasing** the program.

CHAPTER 6

On a real Apple-1: exporting to WAV

Everything you write in the emulator can go to a real Apple-1, a replica or a kit fitted with the ACI cassette card. The audio file replaces the tape; a smartphone or a PC headphone output serves as the tape recorder.

6.1 Making the tape

In the editor, click **Export WAV** and choose a file name. The extension decides the format: `.wav`, or `.aiff` for a phone (the recommended format, see below). The bottom of the editor then shows exactly what to type on the real machine:



Figure 7 — After the export, the bottom of the editor shows the load command.

For `premier.bas`, the command is `C100R` then `004A.00FFR 0F54.0FFFR`. The BASIC program takes two consecutive records on the tape: first BASIC's zero page (`$004A` to `$00FF`, which holds the program pointers), then the program itself, stored at the top of memory. The **Copy the ranges** button puts the command on the clipboard.

> **! ALWAYS FOUR DIGITS**
>
> Type `004A.00FF`, never `4A.FF`. The ACI ROM does not clear the digits already read when it meets the dot: `4A.FF` would be taken as `$004A` to `$4AFF`, and the machine would wait for 19 KB that never come. The emulator always shows the correct form, and it reproduces this behavior because it runs the real ACI ROM.

6.2 Loading on the real machine

BASIC must already be in memory, loaded from its own tape (`C100R` then `E000.EFFFR`).

1. **Prepare the player.** Copy the file to the phone, connect the headphone output to the ACI "FROM TAPE" input, and set the volume to maximum.
2. **Go back to the monitor** with RESET, then type `C100R`. The ACI answers `C100: A9*` and starts a new line; the star is its prompt.
3. **Type the ranges** `004A.00FFR 0F54.0FFFR`, but do not press RETURN yet.
4. **Start playback**, then press **RETURN during the header**, the steady tone of about 10 seconds that opens each record.

5 **Wait for the `\` to come back**, then type `E2B3R` (BASIC warm start, which keeps the program) and `RUN`.

WOZ MONITOR

```
C100R

C100: A9*
004A.00FFR 0F54.0FFFR
\
E2B3R

E2B3: 20
>RUN
HELLO, APPLE-1!
```

▣ READING RELIABLY FROM A PHONE

- **Volume at maximum.** Accept Android's "high volume" warning, otherwise the volume stays capped. Many phones lower the volume every time the jack is plugged in again: check it each time.
- **No sound effects**: turn off the equalizer, Dolby, Mi Sound and any "audio enhancement".
- **Airplane mode** or "Do not disturb", so that no notification mixes into the signal. No Bluetooth.
- **A simple player** that plays AIFF without processing it; VLC works well.
- **Never MP3 or AAC**: compression distorts the signal edges. Stay with WAV or AIFF.
- **Do not trust the ACI LED**: it only lights at about 1.2 V, well above what the card needs.

6.3 Why AIFF at 48 kHz

The emulator's files are designed for the ACI input. Uncompressed 16-bit AIFF escapes every codec; 48 kHz is Android's native rate, so the phone does not resample. Both stereo channels are strictly identical, so left and right can be tied together on FROM TAPE without the two outputs fighting each other. The signal is a square wave with steep edges, which is what the ACI comparator detects best.

✦ CHECKING A FILE BEFORE PLAYING IT

On the command line, `apple1-emu info premier.aiff --aci-input stock` simulates the input of the original card at several voltage levels and tells you from which level the file reads correctly (about 0.25 V peak for our files).

CHAPTER 7

What next?

You now know how to load a tape, write and run a BASIC program, and transfer it to a real machine. The **Complete Manual** (`docs/Complete-Manual.pdf`) goes much further:

- **Integer BASIC guide**: every statement with examples, the limits, the error messages, the known bugs and their workarounds, adapting Applesoft programs.
- **6502 assembler guide**: the built-in assembler (ca65 syntax, macros, listing), the Woz Monitor routines, examples tested in the emulator.
- **Games to play and read**: the library comes with five small games by Claude IA (Hangman, Matches, Battleship in BASIC; Tunnel and Tic-tac-toe in assembly), each in French and English.
- **The tools**: debugger, library and GitHub projects, GEN2 color card and P-LAB graphics card, photo → ASCII, vintage screen, command line.

7.1 Essential shortcuts

Key	Effect
Ctrl+R	RESET (back to the monitor)
Ctrl+L	CLEAR SCREEN
F3	real speed or full speed
Ctrl+V	types the clipboard on the Apple-1 keyboard (also: right-click the screen → Paste)
Ctrl+plus, Ctrl+minus, Ctrl+0	screen zoom, and zoom fitted to the window (in the editor: text size)
F11	full screen
F1	detailed tooltips (like the ? button)
Backspace	`_` , the monitor's rub-out
Ctrl+F, Ctrl+H, Ctrl+G	in the editor: find, replace, go to a line
F5, F9	in the editor: run the tab, breakpoint on the line

7.2 Commands to remember

Command	Effect
`E000R`	starts Integer BASIC (cold: erases the program)
`E2B3R`	returns to BASIC without erasing the program
`C100R`	starts the ACI program (prompt `*`)
`start.endR`	reads the tape into that range, for example `0300.031ER`
`300R`	runs a machine-language program at `$0300`